TUNISIAN REPUBLIC
MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH

UNIVERSITY OF SOUSSE
HIGHER INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY OF SOUSSE



RESEARCH INTERNSHIP REPORT

CLASS: 1ST YEAR ENGINEERING CYCLE

Presented by:

MELKI AYOUB

PPO-Guided Latent Diffusion for Visual Counterfactual Explanations in Chest Radiography (RSNA Dataset)

HOST INSTITUTION:



Maersk Mc-Kinney Moller Institute

Remote Research Internship

Academic Supervisor:

Professor Sadok Ben Yahia

Internship Period:

15/06/2026 – 15/08/2026

Academic Year: 2025–2026

Acknowledgments

I would like to express my sincere gratitude to the Maersk Mc-Kinney Moller Institute at the University of Southern Denmark for hosting this research internship and for granting me the freedom to pursue an open research question rather than a predefined implementation task.

I especially thank my academic supervisor, **Professor Sadok Ben Yahia**, for his guidance, his availability throughout the internship, and for the methodological rigour he insisted upon — in particular the demand that every claimed result be verified against an independent control rather than accepted at face value. That principle shaped the most valuable finding of this work.

I also thank the ISSAT Sousse teaching staff for the academic preparation that made this project possible, and the administrators of the high-performance computing cluster on which every experiment reported here was executed.

Contents

Acknowledgments **ii**

Acronyms **viii**

General Introduction **1**

1 Internship Context Presentation **3**

I Presentation of the Host Institution 3

1 Overview 3

2 Internship Framework 3

II Context and Objectives of the Internship 3

1 Purpose 3

2 Main Objective 4

3 Assigned Work Packages 4

III Expected Results 5

IV Conclusion 5

2 Research and Development Environment **6**

I Computing Infrastructure 6

1 HPC Cluster and the SLURM Scheduler 6

2 GPU Hardware 6

3 Environment Provisioning 7

II Deep Learning Frameworks 7

1 PyTorch 7

2 Diffusers and Stable Diffusion 1.5 7

3 PEFT and LoRA 7

III Pretrained Model Zoo 8

1 TorchXRayVision 8

2 YOLOv8 8

3 Grad-CAM 8

IV Data Access 9

V Toolchain Summary 9

VI Conclusion 9

3 Presentation of the Tasks Performed **10**
I Global Pipeline Architecture . . . 10
1 Delivered Functional Flow . . . 10
2 Splitting Training from Evaluation . . . 10
II Data and Generator Work . . . 11
1 Dataset Preparation . . . 11
2 Class Imbalance and Balanced Sampling . . . 11
3 LoRA Fine-Tuning . . . 11
4 Gradient-Guided DDIM Sampling . . . 11
5 The Background Lock . . . 12
III Reinforcement-Learning Control . . . 12
1 Environment Design . . . 12
2 Action Space . . . 12
3 Actor-Critic Architecture . . . 12
4 Two Directional Policies . . . 13
IV Iterative Debugging and Redesign . . . 13
1 Diagnosis: the Reward Was Almost Entirely a Flip Bonus . . . 13
2 A Reward-Scaling Bug in the Asymmetry Correction . . . 14
3 Grad-CAM Localisation Check . . . 14
4 Sparsity Reward with Annealed Ramp . . . 15
5 Held-Out Validation and the Move to an Ensemble . . . 15
V Experimental Infrastructure . . . 16
1 Smoke Tests . . . 16
2 Job Monitoring . . . 16
VI Conclusion . . . 16

4 Results, Discussion, and Perspectives **17**
I Evaluation Protocol . . . 17
1 Identity Preservation as a Substitute for FOP . . . 17
II Obtained Results . . . 18
1 Primary Metric Suite . . . 18
2 Clinical Proxy Metrics . . . 18
3 Qualitative Counterfactuals . . . 19
4 Training Diagnostics . . . 19
III The Held-Out Validation Result . . . 20
1 Status of the Countermeasures . . . 21
IV Strengths and Limitations . . . 21
1 Strengths . . . 21
2 Limitations . . . 22
V Perspectives and Future Work . . . 22

VI Conclusion 23

General Conclusion **24**

Glossary **25**

Bibliography **27**

List of Figures

3.1 Grad-CAM localisation diagnostic 15

4.1 Counterfactual: normal to pneumonia 19
4.2 Counterfactual: pneumonia to normal 19
4.3 PPO reward trace diagnostic 20

List of Tables

2.1 GPU hardware and measured runtimes 7
2.2 Research and development environment summary. 9

3.1 PPO action space 13
3.2 v1 reward decomposition 14

4.1 Primary evaluation metric suite 18
4.2 Clinical proxy metrics 18
4.3 Held-out validation results 20

Acronyms

- **AI**: Artificial Intelligence
- **BCE**: Binary Cross-Entropy
- **CAM**: Class Activation Map
- **CTR**: Cardio-Thoracic Ratio
- **CV**: Counterfactual Validity
- **CXR**: Chest X-Ray (chest radiograph)
- **DDIM**: Denoising Diffusion Implicit Model
- **DICOM**: Digital Imaging and Communications in Medicine
- **FID**: Fréchet Inception Distance
- **FOP**: Foreign-Object Preservation
- **GAE**: Generalized Advantage Estimation
- **GPU**: Graphics Processing Unit
- **Grad-CAM**: Gradient-weighted Class Activation Mapping
- **HF-corr**: High-Frequency Correlation
- **HPC**: High-Performance Computing
- **IP**: Identity Preservation
- **ISSAT**: Higher Institute of Applied Sciences and Technology of Sousse
- **JSON**: JavaScript Object Notation
- **L1 / L2**: Absolute-value / squared-error norms
- **LoRA**: Low-Rank Adaptation
- **MAE**: Mean Absolute Error
- **MDP**: Markov Decision Process
- **MLP**: Multi-Layer Perceptron
- **MS-SSIM**: Multi-Scale Structural Similarity Index Measure
- **PEFT**: Parameter-Efficient Fine-Tuning
- **PPO**: Proximal Policy Optimization
- **PSPNet**: Pyramid Scene Parsing Network
- **RL**: Reinforcement Learning
- **RSNA**: Radiological Society of North America

- **SCP**: Costophrenic-recess Score (*score costo-phrénique*)
- **SD1.5**: Stable Diffusion version 1.5
- **SDU**: University of Southern Denmark
- **SLURM**: Simple Linux Utility for Resource Management
- **VAE**: Variational Autoencoder
- **VRAM**: Video Random-Access Memory
- **XAI**: Explainable Artificial Intelligence
- **XRV**: TorchXRayVision
- **YOLO**: You Only Look Once (object-detection family)

General Introduction

The transition from theoretical academic study to the practice of scientific research is a decisive stage in an engineer's training. This report documents the work carried out during a research internship completed as part of the first year of the engineering cycle at the Higher Institute of Applied Sciences and Technology of Sousse (ISSAT Sousse), hosted remotely by the Maersk Mc-Kinney Moller Institute at the University of Southern Denmark (SDU).

Deep neural classifiers now reach expert-level accuracy on many medical imaging tasks, but their clinical adoption remains constrained by a question they cannot themselves answer: *why* was this prediction made? The dominant family of post-hoc explanation methods — saliency maps and Grad-CAM among them — answers only *where* the model is looking. It does not say *what* would have to change for the decision to be different. For a radiologist auditing a pneumonia classifier, a heatmap over the right lower lobe is a far weaker statement than an image showing precisely which change to that lobe would flip the model's verdict.

Visual counterfactual explanations address this gap directly. Given a query image x and a target class c, a counterfactual generator produces an image x_c such that a classifier f predicts $f(x_c) \approx c$, while x_c remains a realistic radiograph that preserves the patient's anatomy and any non-pathological content — ribs, mediastinal silhouette, tubes, pacemakers and monitoring leads. The change between x and x_c *is* the explanation.

This internship investigated a specific research hypothesis in that setting: that the per-step control of a guided diffusion sampler should be *learned* rather than fixed. A hand-tuned guidance schedule applies the same editing effort to every image, which under-edits difficult cases and over-edits — and thereby de-realizes — easy ones. The system built during this internship instead frames the choice of per-step sampling parameters as a sequential decision problem and trains a Proximal Policy Optimization (PPO) agent to solve it, on top of a Stable Diffusion 1.5 latent generator adapted to chest radiography with Low-Rank Adaptation (LoRA).

The work was organised around four research questions:

- Can a reinforcement-learning agent that controls only the *sampling procedure* — never the diffusion weights — produce valid counterfactuals on chest radiographs?
- Can the edits be spatially confined to clinically plausible regions, rather than diffusing over the whole image?
- Do the resulting counterfactuals encode *genuine disease signal*, or do they merely constitute adversarial perturbations against the particular classifier used for guidance?
- How do the standard counterfactual evaluation metrics of Singla *et al.* [1] behave under the compute constraints of a two-month single-GPU project?

The third question turned out to be the scientifically decisive one, and its answer reshaped the second half of the internship. An independent held-out validation — re-scoring the generated counterfactuals with classifiers that had never participated in the guidance loop — showed that a system

reporting near-perfect validity against its own guidance classifier transferred very poorly to independent models. That negative result, and the redesign it motivated, form the substantive contribution of this work and are reported in full in Chapter 4.

The report is organised into four chapters:

- **Chapter 1** presents the internship context, the host research institution, and the objectives, work packages and expected outcomes of the project.
- **Chapter 2** describes the research and development environment: the HPC cluster and its SLURM scheduler, the deep-learning frameworks, the pretrained model zoo, and the dataset-access tooling.
- **Chapter 3** details the tasks performed — the architecture of the pipeline, its module-by-module construction, and the iterative debugging and redesign cycles that the experimental evidence forced.
- **Chapter 4** reports the measured results, discusses the strengths and the substantial limitations of the current system, and sets out realistic perspectives for future work.

Chapter 1

Internship Context Presentation

This chapter establishes the context of the internship, introduces the host research institution, and sets out the objectives, work packages and expected outcomes of the project.

I Presentation of the Host Institution

1 Overview

The internship was carried out at the **Maersk Mc-Kinney Moller Institute**, a research institute of the **University of Southern Denmark (SDU)**. The Institute is an academic research environment organised around robotics, artificial intelligence, embedded systems and applied data science, and its work is structured around research questions and publications rather than around product delivery for clients.

This distinction shaped the entire internship. There was no client, no specification document and no delivery deadline for a feature set. The deliverable was a defensible answer to a research question, together with the experimental apparatus needed to reproduce it — which, as Chapter 4 discusses, includes the apparatus needed to *disprove* an initially promising result.

2 Internship Framework

This work fulfils the research internship requirement of the first-year engineering cycle at ISSAT Sousse. The internship was conducted **remotely**, under the academic supervision of **Professor Sadok Ben Yahia**, over the period 15 June 2026 to 15 August 2026.

The remote format had a concrete methodological consequence. All computation ran as batch jobs on a high-performance computing cluster under the SLURM scheduler, with no interactive session and no ability to attach a debugger to a running job. Every experiment therefore had to be specified completely in advance, instrumented well enough to be diagnosed from its log file alone, and cheap enough to fail fast. This constraint is visible throughout the engineering choices described in Chapter 3.

II Context and Objectives of the Internship

1 Purpose

Explainable Artificial Intelligence (XAI) for medical imaging is dominated by attribution methods, which localise evidence but do not describe the change that would alter a decision. Counterfactual image generation supplies that missing description, but it introduces a verification problem of its

own: an image that flips a classifier's output has not necessarily acquired — or lost — any disease. It may simply have acquired an adversarial perturbation tailored to that one model.

The purpose of this project was therefore twofold:

- To build a counterfactual generation system for binary pneumonia classification on chest radiographs, in which the diffusion sampling schedule is *learned* by a reinforcement-learning agent rather than fixed by hand.
- To evaluate that system honestly — including with controls capable of exposing the adversarial failure mode — rather than reporting only the metric that the system was optimised against.

2 Main Objective

The central objective was to engineer and evaluate a complete research pipeline that, given a chest radiograph x and a target class c, produces a counterfactual x_c satisfying three simultaneous criteria:

1. **Validity**: a frozen classifier f scores x_c at the target class with a large margin.
2. **Realism**: x_c is distributionally indistinguishable from genuine radiographs.
3. **Identity preservation**: x_c differs from x only in a minimal, class-relevant region, leaving patient anatomy and foreign objects intact.

The methodological commitment underlying the whole system is that the diffusion model's *weights* are never optimised by reinforcement learning. The PPO agent controls only the sampling procedure — noise strength, guidance scales, number of refinement passes, and the stopping decision — around a frozen, LoRA-adapted generator. This keeps the action space small and interpretable, and makes the learned policy reusable across arbitrary query images at inference time.

3 Assigned Work Packages

The project was decomposed into six work packages:

- **WP1 — Data ingestion.** Programmatic acquisition of the RSNA Pneumonia Detection Challenge dataset, DICOM decoding, normalisation, and derivation of a binary label from the bounding-box annotations.
- **WP2 — Pretrained model integration.** Loading and freezing a pneumonia classifier, a lung/heart segmentation network, and an object-detector feature extractor, all serving as fixed sources of guidance and reward signal.
- **WP3 — Generator adaptation.** Parameter-efficient LoRA fine-tuning of a Stable Diffusion 1.5 UNet from the natural-image domain to chest radiography.
- **WP4 — Reinforcement-learning control.** Design of a Gym-style environment, a shaped reward function, and a PPO actor-critic agent; training of two directional policies, one per class-flip direction.
- **WP5 — Evaluation.** Implementation of the Singla *et al.* [1] metric suite (CV, FID, FOP) plus clinical proxy metrics, and of pixel-level difference visualisation.

- **WP6 — Validation and diagnostics.** Independent held-out re-scoring to distinguish genuine disease signal from adversarial perturbation, and Grad-CAM analysis of the guidance classifier for shortcut learning.

WP6 was not part of the initial plan. It was added mid-internship, in direct response to the observation that the primary validity metric was computed with the same classifier that had produced the guidance gradient, and was therefore circular. Its results are the central finding of this report.

III Expected Results

The anticipated deliverables at the outset of the internship were:

- A reproducible, end-to-end research pipeline covering ingestion, generator adaptation, policy training and evaluation, executable as batch jobs on an HPC cluster.
- Two trained directional policies together with their checkpoints and configuration provenance.
- A quantitative evaluation under the reference metric protocol, with the numbers traceable to serialised artefacts rather than to transcription.
- A critical assessment of what the system does and does not demonstrate, including its failure modes.

IV Conclusion

This chapter has set out the academic research context of the internship, introduced the Maersk Mc-Kinney Moller Institute as host institution, and defined the objectives and work packages of the project. The following chapter details the research and development environment — the cluster, the frameworks, and the pretrained model zoo — within which this work was carried out.

Chapter 2

Research and Development Environment

This chapter presents the computational infrastructure, software frameworks, pretrained models and data-access tooling used to design, train and evaluate the system. Only components actually used in the project are described; version numbers are those pinned in `requirements.lock.txt`.

I Computing Infrastructure

1 HPC Cluster and the SLURM Scheduler

All training and evaluation ran on a shared high-performance computing cluster managed by **SLURM** (Simple Linux Utility for Resource Management) [15], an open-source batch scheduler that allocates compute nodes to queued jobs according to resource requests and fair-share priority.

Every entry point of the project has a corresponding batch script (`run_ppo.slurm`, `run_eval_ppo.slurm`, `run_heldout_validation.slurm`, `run_classifier.slurm`, `run_smoke_test.slurm`, `run_smoke_test_eval.slurm`). Each requests one GPU, 8 CPU cores and 64 GB of system memory on the `publicgpu` partition, and differs mainly in its walltime request — a parameter that, as Section 2 of Chapter 3 describes, turned out to matter a great deal.

Two properties of this environment drove most of the engineering decisions in Chapter 3:

- **Compute nodes have no internet access.** Any model weight or dataset must be downloaded in advance on the login node. All batch scripts therefore export `HF_HUB_OFFLINE=1`, `TRANSFORMERS_OFFLINE=1` and `YOLO_OFFLINE=1`, converting what would otherwise be a silent network hang into an immediate, legible failure.
- **Jobs are non-interactive and are killed at walltime.** A job that exceeds its requested time limit is terminated unconditionally, whatever it was doing.

2 GPU Hardware

The cluster is heterogeneous, and the GPU model assigned to a job depends on which node the scheduler allocates. Two devices were observed across this project's runs, with measured throughput recorded in the batch scripts:

The generator's own memory footprint is modest — the pipeline logs a VRAM estimate of 1826 MB for the VAE and UNet combined — so the 16 GB devices available were never the binding constraint. Wall-clock time was.

Table 2.1: GPU hardware observed across project runs, with measured wall-clock cost of the two main pipeline halves (figures recorded in `run_ppo.slurm` and `run_eval_ppo.slurm`).

Device	VRAM	PPO training	Evaluation
Tesla P100-PCIE	16 GB	$\sim$20 h	$>$4 h (unfinished)
Quadro RTX 5000	16 GB	$\sim$12.5 h	$\sim$2.5 h

3 Environment Provisioning

`setup_env.sh` is a login-node script that creates the Python virtual environment, installs the pinned dependency set, and — most importantly — *prefetches* every pretrained weight the compute nodes will later need, including the several TorchXRayVision DenseNet checkpoints. Dependencies are tracked in two files: `requirements.txt` declares loose minimum versions for readability, while `requirements.lock.txt` pins exact versions (`torch==2.5.1+cu121`, `diffusers==0.38.0`, `peft==0.19.1`, `torchxrayvision==1.5.2`, `ultralytics==8.4.83`) so that a run months later reproduces the same numerical environment.

II Deep Learning Frameworks

1 PyTorch

PyTorch 2.5.1 with CUDA 12.1 is the numerical foundation of the entire project. Its define-by-run autograd is not merely a convenience here but a structural requirement: the counterfactual guidance mechanism differentiates the frozen classifier's loss *with respect to an intermediate latent inside the diffusion sampling loop*, a gradient path that must be constructed dynamically at each denoising step. Models run in `float16` on GPU (`config.DTYPE`), with selected parameters upcast to `float32` where numerical stability requires it.

2 Diffusers and Stable Diffusion 1.5

The generator is built on **Stable Diffusion 1.5** (`runwayml/stable-diffusion-v1-5`), a latent diffusion model [4] loaded through the Hugging Face `diffusers` library. Rather than training a diffusion model from scratch on a comparatively small medical dataset, the project reuses SD1.5's pretrained VAE and UNet and adapts only a small parameter subset.

Images of 256×256 pixels are encoded by the frozen VAE into a $4 \times 32 \times 32$ latent tensor, an $8\times$ spatial compression that reduces the cost of every subsequent sampling and guidance step by two orders of magnitude relative to pixel-space diffusion. Sampling uses **DDIM** [5], whose deterministic, non-Markovian reverse process permits a partial forward-then-reverse trajectory — the mechanism by which an existing radiograph is edited rather than a new image synthesised from noise.

3 PEFT and LoRA

Full fine-tuning of the SD1.5 UNet (866 million parameters) was out of scope for the available compute. **LoRA** [6], applied through the Hugging Face `peft` library, instead freezes the pretrained weights and injects trainable low-rank matrix pairs into selected projections — here the attention layers' `to_q`, `to_k`, `to_v` and `to_out.0` (`config.LORA_TARGETS`), at rank 16 with $\alpha = 16$ and dropout 0.05.

The pipeline log records the resulting parameter economy exactly: **6,782,976 trainable parameters out of 866,303,940**, or **0.78%** of the model. This is what made domain adaptation feasible within the internship's compute budget.

III Pretrained Model Zoo

The system depends on three frozen pretrained networks, none of which is ever updated by training. They supply, respectively, the guidance gradient, the spatial constraint, and part of the preservation signal.

1 TorchXRayVision

TorchXRayVision (XRV) [10] is a library of chest X-ray datasets and pretrained models with a standardised preprocessing convention. It supplies two distinct components here:

- **Classifiers** — DenseNet-121 models trained on different chest X-ray corpora and exposed under a common interface (`densenet121-res224-rsna`, `-chex`, `-nih`, `-mimic_ch`, `-pc`). The availability of *multiple independently trained* classifiers over a shared label space is what makes the held-out validation of Chapter 4 possible at all; this is the single most consequential tooling choice in the project.
- **Segmentation** — a **PSPNet** providing per-anatomy masks. The lung channels gate where editing is permitted; the heart and lung channels together yield the Cardio-Thoracic Ratio used as a clinical proxy metric.

2 YOLOv8

YOLOv8 [14] (nano variant, weights in `yolov8n.pt`, loaded via `ultralytics`) is used not as a detector but as a *frozen feature extractor*. Its backbone embedding of an image serves as a coarse proxy for structural and foreign-object content: penalising drift in this embedding discourages the generator from destroying tubes, leads and other hardware. It also underlies the costophrenic-recess proxy metric.

The report is explicit that this is an approximation. YOLOv8n is pretrained on COCO, a natural-image corpus, and is not a radiograph-specific device detector; Section 2 discusses the consequences.

3 Grad-CAM

Grad-CAM [9] produces class-discriminative localisation maps by weighting convolutional feature maps with the gradients flowing into them. A single shared implementation (`src/gradcam.py`) serves two distinct roles in this project:

- **Diagnostic** — `check_classifier_localization.py` renders Grad-CAM overlays to check whether the guidance classifier attends to lung pathology or to a global brightness/contrast shortcut.
- **Constraint** — the same map, thresholded and feathered, restricts the region in which the PPO agent is permitted to edit (Chapter 3, Section 3).

IV Data Access

The **RSNA Pneumonia Detection Challenge** dataset [11] is retrieved programmatically through `kagglehub` (`ingestion.py`), which handles authentication and local caching so that a compute node can later read the data with no network access. Images arrive as DICOM and are decoded with `pydicom` (with `pylibjpeg` for JPEG-compressed transfer syntaxes), min–max normalised to $[0, 1]$ and resized to 256×256.

The ingestion log records the corpus actually loaded: **26,684 studies**, of which **6,012 are pneumonia-positive** and **20,672 negative** — a 22.5% positive rate whose consequences for training are discussed in Chapter 3.

V Toolchain Summary

Table 2.2: Research and development environment summary.

Category	**Tools / Technologies**
Compute infrastructure	SLURM batch scheduler; Tesla P100-PCIE-16GB and Quadro RTX 5000 GPU nodes; offline (no-internet) compute nodes
Numerical framework	PyTorch 2.5.1 + CUDA 12.1, `float16` inference
Generative model	Stable Diffusion 1.5 via `diffusers` 0.38.0; DDIM sampling
Parameter-efficient tuning	LoRA via `peft` 0.19.1 (rank 16, α 16)
Pretrained medical models	TorchXRayVision 1.5.2 (DenseNet-121 classifiers, PSPNet segmenter)
Object-detection features	Ultralytics YOLOv8n 8.4.83
Explainability	Grad-CAM (custom shared implementation)
Data access	`kagglehub`, `pydicom`, `pylibjpeg`
Scientific stack	NumPy, SciPy, scikit-image, scikit-learn, Matplotlib
Version control	Git

VI Conclusion

This chapter has described the computational environment of the internship: a non-interactive, offline, walltime-bounded HPC cluster, a PyTorch-based generative stack, and a set of frozen pretrained medical models. The next chapter details how these components were assembled into a working research pipeline, and how experimental evidence repeatedly forced that pipeline to be redesigned.

Chapter 3

Presentation of the Tasks Performed

This chapter details the technical work carried out during the internship: the architecture of the research pipeline, the construction of each of its modules, and — occupying the larger part of the internship — the successive redesign cycles that experimental evidence forced upon it.

I Global Pipeline Architecture

1 Delivered Functional Flow

The system is organised as a single linear data flow, split across two independent executable entry points that share a common construction layer:

1. `ingestion.py` downloads the RSNA dataset and exposes it as an `RSNACXRDataset`.
2. `pipeline.py`'s `build_common()` constructs the dataset, the frozen models and the generator. Both entry points call it, which guarantees that training and evaluation operate on *identical* objects.
3. `src/models.py` loads and freezes the classifier, the PSPNet segmenter and the YOLOv8 feature extractor.
4. `src/generator.py` and `factory/finetuning.py` adapt the Stable Diffusion UNet to the radiograph domain via LoRA.
5. `factory/environment.py` and `factory/ppo.py` train two directional PPO policies.
6. `eval_main.py` reloads the checkpoints and runs `src/metrics.py` and `src/diff_viz.py`.

Alongside this main path sit two diagnostic tools that never participate in training: `heldout_validation.py` and `check_classifier_localization.py`.

2 Splitting Training from Evaluation

The pipeline was originally a single process, `main.py`, that trained both policies and then evaluated them. This design failed concretely. Job **17644171** completed PPO training, wrote both policies to disk, and was then killed at the 24-hour walltime boundary partway through evaluation. No metrics were produced, and — critically — no entry point existed that could reload the surviving policies. A full training run had to be repeated to recover a number that the finished policies could already have produced.

The pipeline was restructured in response. `main.py` now performs Steps 1–5 (training) and terminates after writing checkpoints; `eval_main.py` performs Steps 6–6B (evaluation) as a separate process, failing fast with an explicit message if the expected policy checkpoints are absent. Beyond removing the walltime hazard, this gives evaluation the property of being freely repeatable: metric fixes, different sample sizes and regenerated plots no longer cost a training run.

This split also created a subtler risk, addressed by `smoke_test_eval.py`: with training and evaluation in one process, a checkpoint that fails to reload correctly in a *cold* process is invisible, because the objects are already in memory. The evaluation smoke test exercises precisely that cold-reload path.

II Data and Generator Work

1 Dataset Preparation

`ingestion.py` retrieves the RSNA Pneumonia Detection Challenge corpus and derives a binary label from the challenge's bounding-box annotations: a study is positive if any box marks lung opacity, negative otherwise. DICOM files are decoded, min–max normalised to $[0, 1]$ and resized to 256×256. The loaded corpus comprises **26,684 studies**: 6,012 positive and 20,672 negative.

2 Class Imbalance and Balanced Sampling

The 22.5% positive rate is a problem for denoising fine-tuning, which is class-conditional: trained on the natural distribution, the generator sees pneumonia-positive radiographs less than a quarter of the time and learns a correspondingly weaker conditional model for exactly the class the harder edit direction must synthesise.

`factory/finetuning.py` addresses this with a `WeightedRandomSampler` (`_build_balanced_sampler`), weighting each sample by the inverse frequency of its class so that each epoch presents roughly balanced classes. The function fails loudly rather than silently degrading if a class is absent from the dataset.

3 LoRA Fine-Tuning

The SD1.5 UNet is adapted with rank-16 LoRA adapters ($\alpha = 16$, dropout 0.05) inserted into the attention projections `to_q`, `to_k`, `to_v` and `to_out.0`, trained on the standard ϵ-prediction denoising objective for 30 epochs at learning rate 10^{-4}. Class conditioning replaces SD1.5's text encoder entirely: a small `cond_proj` MLP maps the target class and a pooled latent summary to a single cross-attention conditioning token.

The training loop checkpoints every epoch and resumes from `checkpoints/finetuning_latest.pt`, a necessity in a walltime-bounded batch environment — the log of the most recent run shows it correctly detecting a completed 30-epoch fine-tune and skipping straight past Step 4.

An earlier iteration used rank 8; its checkpoints are preserved under `checkpoints/archive_v1_rank8/` for comparison.

4 Gradient-Guided DDIM Sampling

Counterfactual generation (`src/generator.py`) performs a partial forward-then-reverse DDIM trajectory, whose length is set by a `strength` fraction of the $T_{\text{diff}} = 50$ step schedule. At each reverse step, two gradient terms are injected into the predicted clean latent $\hat{z}_0$:

$$\Delta z_0 = -\,w(i)\,\alpha_{\text{cls}}\,\nabla_{\hat{z}_0}\mathcal{L}_{\text{cls}} \;-\; w(i)\,\alpha_{\text{pres}}\,\nabla_{\hat{z}_0}\big(\text{seg-weighted L1} + \eta\,\text{obj-det MSE}\big)$$

The first term drives the classifier toward the target class; the second penalises departure from the original image, both in anatomy (segmentation-weighted L1) and in object-detector feature space. The factor $w(i)$ is a linear ramp that rises from 0 to 1 starting at a policy-chosen fraction of the trajectory, replacing an earlier hard on/off guidance switch whose discontinuity destabilised sampling.

5 The Background Lock

Gradient guidance alone provides no guarantee about *where* an edit lands. The generator therefore applies a hard spatial constraint adapted from RePaint [7] and Blended Latent Diffusion [8]: at every denoising step, the latent *outside* a dilated, Gaussian-feathered lung-segmentation mask is overwritten with the original image's latent, forward-noised to the current timestep (and clean at the final step).

This is a structural guarantee rather than a soft penalty. Content outside the lung field cannot be repainted by the sampler regardless of how the guidance gradient behaves; it can only collapse to the VAE's reconstruction floor. Without it, an aggressive `strength` schedule repaints the entire thorax.

III Reinforcement-Learning Control

1 Environment Design

`MedicalCounterfactualEnv` (`factory/environment.py`) exposes the sampling procedure as a Gym-style MDP. An episode begins by sampling a "far" image — one the classifier scores more than `FAR_THRESHOLD`$= 0.5$ away from the target class. The most recent run logs the resulting pool: **15,267 of 26,684 images** qualify for the n→p direction, split into 15,243 for rollouts and 24 withheld for a validity probe.

The observation concatenates the flattened original and current latents, a normalised step index, the current $|f(x_t) - c|$, the target class, and the flattened lung mask — 73,731 dimensions in total, as logged at policy construction. Episodes terminate when $|f(x_t) - c| < \varepsilon_{\text{stop}} = 0.08$ or the step budget $T = 8$ is exhausted.

2 Action Space

The policy emits a 9-dimensional continuous action at each step, mapped through sigmoid transforms onto bounded control ranges:

The classifier-guidance ceiling in a_1 was deliberately narrowed during development, from a maximum of 3.0 down to 1.2. At the original ceiling a single step could brute-force the classifier flip, and episodes collapsed to an average length of exactly 1.00 — leaving PPO with no temporal structure from which to learn credit assignment. Capping the per-step push forces the flip to be reached over several steps, which is what makes the problem a sequential decision problem at all.

3 Actor-Critic Architecture

`PPOActorCritic` (`factory/ppo.py`) uses a convolutional trunk over the latent-shaped portion of the observation (three `Conv2d` layers, $32 \to 64 \to 128$ channels, followed by a 256-unit

Table 3.1: PPO action space (`factory/environment.py`, `MedicalCounterfactualEnv.step`).

Dim	Controls	Range
a_0	DDIM strength	$[0.20,\ 0.70]$
a_1	classifier-guidance scale	$[0.30,\ 1.20]$
a_2	preservation-guidance scale	$[0.30,\ 1.50]$
a_3	stop signal τ_t	$[0,\ 1]$
a_4, a_5	one extra refinement pass each (Bernoulli gates)	$\{0, 1\}$
a_6	per-pass strength boost	$[0.85,\ 1.15]$
a_7	guidance bias (classifier vs. preservation)	$[0.70,\ 1.30]$
a_8	guidance ramp-up start fraction	$[0.10,\ 0.50]$

MLP head that also consumes the scalar observation components). This trunk feeds four heads: a Gaussian policy head with a **learnable per-dimension `log_std`** parameter, a value head, and three auxiliary heads predicting feasibility, progress and the stop decision, each trained with its own auxiliary loss.

Making `log_std` a learned parameter rather than a fixed schedule lets the agent modulate its own exploration per action dimension. The resulting policy is compact: **197,910 parameters**, against an observation of $73{,}731$ dimensions.

Advantages are computed with GAE [3] ($\gamma = 0.99$, $\lambda = 0.95$) and the policy updated with the clipped PPO objective [2] (clip 0.2, value coefficient 0.5, entropy coefficient 0.01).

4 Two Directional Policies

The environment is constructed around a single fixed target class, so two policies are trained independently: one for normal→pneumonia ($c = 1$) and one for pneumonia→normal ($c = 0$). They share the same fine-tuned generator but specialise their sampling schedules. A single target-conditioned policy would be architecturally cleaner, and is identified in Section V as future work.

IV Iterative Debugging and Redesign

The chronology below is reconstructed from the repository's commit history and from the design rationale recorded in `config.py`, which was maintained as a running experimental log throughout the internship.

1 Diagnosis: the Reward Was Almost Entirely a Flip Bonus

Early counterfactuals changed between 66% and 75% of all pixels — diffuse, whole-image edits rather than localised lesions. Decomposing the reward actually obtained by the v1 policy (reconstructed from job **17648782**, which logged `avg_return`$= 33.931$ at `avg_ep_len`$= 1.00$) explained why:

The flip bonus *was* the reward. Preservation contributed roughly one part in a hundred and image fidelity one part in ten thousand, so the optimal policy was to flip the classifier by any means available and stop immediately. Three changes followed:

- **Rebalancing.** The terminal bonuses were cut (λ_{hit} and λ_{stop} from 6 and 8 down to 2 each) and the preservation weight raised (λ_{pres} from 0.5 to 3.0), targeting a roughly 40/40/20 split between flip,

Table 3.2: Reward decomposition of the v1 policy, reconstructed from job 17648782. The terminal flip bonuses accounted for 93% of the return; preservation was numerically negligible.

Reward term	Contribution	Share
R_{hit} $\times 6$ and R_{stop} $\times 8$	$+14.00$	93%
R_{prog} $\times 1.5$	$+1.40$	9%
R_{pres} $\times 0.5$	-0.21	-1.4%
R_{fid} $\times 0.5$ (on MSE 0.0033)	-0.0016	-0.01%

preservation and progress.

- **L2 → L1 fidelity penalty.** R_{fid} was computed as a mean squared error. L2 prefers many small changes over few large ones — it was *actively rewarding* the diffuse failure mode being diagnosed. Switching to L1, which is sparsity-inducing, required rescaling λ_{fid} from 0.5 to 2.0, since L1 is roughly $25\times$ larger numerically at these magnitudes.
- **Episode-origin anchoring.** With `ANCHOR_TO_EPISODE_ORIGIN`, the background lock and the preservation targets are anchored to the episode's *original* image rather than to the previous refinement pass's output. Anchoring to the previous pass allows unbounded drift over an episode; anchoring to the origin restores the Markov property and forces the policy to refine rather than to blast.

2 A Reward-Scaling Bug in the Asymmetry Correction

The n→p direction is empirically harder than p→n, and was originally compensated with a $2.25\times$ `direction_scale` boost on its reward. The scale was found to be applied only to the progress terms and not to R_{fid} or R_{pres} — meaning preservation was effectively $2.25\times$ *less* important on precisely the direction that was already producing the most diffuse edits.

Applying the scale uniformly across all reward terms corrected the relative weighting. It is worth stating explicitly, as the code's own change log does, that under PPO's per-update advantage normalisation a uniform scalar multiplying the entire reward is close to a no-op for the policy gradient. The fix removed a harmful *relative* re-weighting; it did not inject additional learning signal into the harder direction, and the residual asymmetry in Chapter 4 is consistent with that.

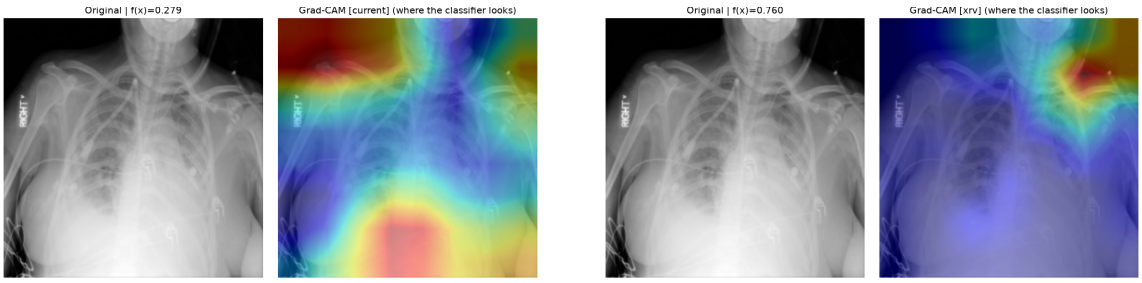
3 Grad-CAM Localisation Check

Before trusting a classifier to supply a guidance gradient, it is necessary to establish that it responds to pathology rather than to a global brightness or contrast shortcut — a well-documented failure mode of CXR classifiers, and one that would make every generated counterfactual a mere exposure adjustment.

`check_classifier_localization.py` renders Grad-CAM overlays for this purpose. The comparison between the original classifier and the TorchXRayVision DenseNet-121 is shown in Figure 3.1.

The same Grad-CAM implementation was subsequently promoted from a diagnostic to a *constraint*. With `USE_GRADCAM_MASK`, the map is normalised at its 99th percentile, raised to a power $\gamma = 0.75$, floored at 0.15 and Gaussian-blurred, then used to modulate the edit budget spatially — so the agent may edit freely where the classifier attends and only marginally elsewhere.

Enabling this required an ensemble-aware Grad-CAM path (`gradcam.ensemble_grad_cam`). Without it, resolving the target layer on an ensemble walks `named_modules()` and silently re-



(a) Original classifier. **(b)** TorchXRayVision DenseNet-121.

Figure 3.1: Grad-CAM localisation diagnostic, one representative case from the six pairs written to `outputs/gradcam_check/`. This diagnostic motivated the replacement of the project's own ResNet-18 classifier (`checkpoint/classifier.pt`) with the pretrained TorchXRayVision DenseNet-121 as the guidance model.

turns the *first member's* feature block, producing a single-model CAM mislabelled as the ensemble's. This is recorded in `config.py` as verified behaviour rather than a suspicion.

4 Sparsity Reward with Annealed Ramp

Even after the reward rebalance, measured edit concentration — the share of total $|\Delta|$ carried by the top 5% of pixels — stood at 0.282 (n→p) and 0.297 (p→n) against a uniform baseline of 0.05. A concentrated core existed, but most of the edit mass sat in a diffuse tail.

An explicit L1 sparsity reward term was added to target that tail, with two refinements. First, it is *annealed*: the weight is zero for the first 20% of updates, ramps linearly to its target of 2.0 between 20% and 60%, and is constant thereafter, so the policy first learns a reliable flip and only then is pressured toward minimal edits. Second, with `SPARSITY_OUTSIDE_CAM_ONLY` the penalty is weighted by $(1 - \text{CAM})$, penalising only edit mass *outside* the attended region — a plain L1 would also penalise the concentrated core that the method is trying to keep.

A safeguard accompanies the ramp: validity is probed on a held-out set every 10 updates, and the ramp freezes if it drops more than 8 points below its pre-ramp baseline. The size of that held-out set was chosen rather than guessed. Validity is a fraction over N images, so its resolution is $1/N$; at $N = 8$ a single image moves it by 12.5 points and the freeze would latch on noise in essentially every run. $N = 24$ gives 4.2 points per image, placing the 8-point tolerance safely above the noise floor.

5 Held-Out Validation and the Move to an Ensemble

The most consequential finding of the internship came from questioning the primary metric. Counterfactual Validity was being computed with the *same* classifier that produced the guidance gradient. That number is circular: it measures how well an optimiser optimised its own objective, not whether the resulting image depicts disease.

`heldout_validation.py` was written as the independent check. It re-scores the generated counterfactuals with TorchXRayVision classifiers that never participated in the guidance loop, and includes an agreement control on the *original* images — so that poor transfer cannot be dismissed as the validators simply disagreeing about the task. It is deliberately a separate SLURM job rather than part of `eval_main.py`, and caches its generated counterfactuals so that adding a validator costs minutes rather than hours.

The results (Chapter 4, Section III) were negative, and led directly to a redesign: the guidance

signal was changed from the single RSNA model to an **ensemble** of `densenet121-res224-rsna` and `densenet121-res224-chex`. The reasoning is that an adversarial direction found against one model rarely survives against an independently trained one, so requiring consensus makes the adversarial shortcut a substantially narrower target. The `-chex` model was chosen specifically because its 11-head multi-task training resists the global-brightness shortcut.

This change carries a bookkeeping obligation recorded in the configuration: any classifier in the guidance ensemble ceases to be a valid held-out validator. The validator set was accordingly reduced to `nih`, `mimic_ch` and `pc`.

V Experimental Infrastructure

1 Smoke Tests

`smoke_test.py` and `smoke_test_eval.py` run the training and evaluation paths end-to-end with deliberately tiny budgets, to surface crashes and tensor-shape errors before a 20-hour job is queued. The training smoke test is parameterised by an `SMOKE_ARM` environment variable selecting `baseline` (all new flags off, reproducing the pre-change pipeline), `mask` (Grad-CAM constraint only) or `all` — the same ablation arms used to attribute behaviour changes to individual mechanisms.

2 Job Monitoring

All batch scripts declare `-mail-type=END,FAIL` with a notification address, so job completion and failure are reported without polling. Each script also runs `set -euo pipefail` and performs explicit precondition checks before loading any model weights — the evaluation and held-out scripts verify that the expected policy checkpoints exist, and the held-out script additionally verifies that each validator's weights are present in the local TorchXRayVision cache, failing in seconds rather than after three hours of generation.

One such check encodes a non-obvious detail worth recording: the TorchXRayVision cache filename does not always match the weight identifier. `densenet121-res224-rsna` is cached as `kaggle-densenet121-*.pt`, because RSNA was a Kaggle competition, so a naive existence check for `rsna-densenet121-*` fails even on a correctly provisioned machine.

Python is invoked unbuffered (`python -u`) throughout, so that a job's log is readable while it is still running. In a non-interactive environment where a stalled job cannot be inspected directly, the log is the only diagnostic channel available.

VI Conclusion

This chapter has described the construction of the research pipeline and the sequence of redesigns that experimental evidence forced: a reward function found to be 93% flip bonus, a fidelity penalty that rewarded the failure mode it was meant to suppress, a direction-scaling bug that penalised preservation on the harder direction, and — most importantly — a primary metric found to be circular. The next chapter reports what the resulting system measures, and what those measurements do and do not establish.

Chapter 4

Results, Discussion, and Perspectives

This chapter reports the measurements obtained, assesses what they do and do not establish, and sets out perspectives for continuing the work. All figures quoted are read directly from the serialised evaluation artefacts in `checkpoints/` — principally `eval_metrics.json`, `heldout/heldout_validation.json` and the two `guidance_diagnostics_*.json` files — rather than transcribed from run logs.

I Evaluation Protocol

Following Singla *et al.* [1], the evaluation set is split by classifier confidence into a negative group $X_n = \{x : f(x) < 0.2\}$ and a positive group $X_p = \{x : f(x) > 0.8\}$, with up to $n_{\text{eval}} = 200$ images each. The n→p policy generates counterfactuals from X_n toward $c = 1$; the p→n policy generates from X_p toward $c = 0$. Both use the policy's deterministic mean action, for reproducibility.

The metric suite comprises Counterfactual Validity (CV), Fréchet Inception Distance (FID) [13] computed in TorchXRayVision DenseNet feature space rather than ImageNet Inception space, a model-free Identity Preservation (IP) score, and two clinical proxy metrics — Cardio-Thoracic Ratio (CTR) and a costophrenic-recess score (SCP) — each tested with a paired t-test on the within-image delta.

1 Identity Preservation as a Substitute for FOP

The reference protocol's Foreign-Object Preservation metric requires a Fast R-CNN device detector trained on private, manually annotated data. It is not reproducible offline, and no TorchXRayVision weight set exposes a usable "Support Devices" head. FOP is accordingly reported as **unavailable** — it is serialised as `null` in `eval_metrics.json`, not silently omitted — and substituted by a model-free IP score measuring the same underlying property:

- **MS-SSIM** [12]: multi-scale structural similarity between x and x_c, capturing global anatomical identity.
- **HF-corr**: Pearson correlation between high-pass-filtered versions of x and x_c. Thin, high-contrast structures — tubes, leads, pacemakers — are exactly the spatial-frequency content a Gaussian high-pass isolates, making this a direct proxy for whether such hardware survives editing.

Table 4.1: Evaluation results from `checkpoints/eval_metrics.json`, $n_{\text{eval}} \leq 200$ per group. FID is computed between each real group and its own counterfactual set.

Metric	Definition	n→p	p→n
CV	fraction valid ($\tau = 0.8$)	0.995	0.955
	mean	0.975	
FID	XRV-DenseNet features, lower better	103.83	59.73
IP	MS-SSIM	0.849	0.796
IP	HF-corr	0.938	0.798
FOP	not computable offline	—	—

II Obtained Results

1 Primary Metric Suite

Counterfactual Validity is high in both directions. Section III explains why this number, taken alone, is not evidence of success.

Identity preservation is respectable and consistently better for the n→p direction (MS-SSIM 0.849, HF-corr 0.938) than for p→n (0.796, 0.798), indicating that fine high-frequency structure — the hardware the HF-corr proxy is designed to track — survives editing substantially better when adding a pattern than when removing one.

The FID values are high in absolute terms. Two factors compound here. The estimates use $N \approx 200$ samples against a 1024-dimensional feature space, far below the sample count FID nominally requires, so their variance is large. Beyond that, they indicate a genuine distributional gap between generated and real radiographs that the current training budget did not close. These should be read as directionally informative — p→n is clearly the more realistic direction — rather than as tightly bounded estimates.

2 Clinical Proxy Metrics

Table 4.2: Clinical proxy metrics: group means ± standard deviation and paired t-tests on the within-image real→counterfactual delta. Significance at $\alpha = 0.05$ marked with *.

	Real group	Counterfactual	Δ (t, p)
n→p direction ($X_n \to X_c^{n \to p}$)			
CTR	0.424 ± 0.055	0.417 ± 0.078	$+0.0070$ ($t = 1.09$, $p = 0.276$)
SCP	0.4794 ± 0.0008	0.4795 ± 0.0009	-0.00014 ($t = -2.89$, $p = 0.0042^*$)
p→n direction ($X_p \to X_c^{p \to n}$)			
CTR	0.452 ± 0.101	0.452 ± 0.131	$+0.0251$ ($t = 2.22$, $p = 0.028^*$)
SCP	0.4791 ± 0.0010	0.4792 ± 0.0011	-0.00012 ($t = -1.52$, $p = 0.129$)

The picture here is mixed and should not be overstated. Only two of the four tests reach significance, and they are not the same two: CTR shifts significantly in the p→n direction but not in n→p, while SCP does the reverse.

The CTR result is arguably the more interpretable of the two, and it is interpretable as a *success of the spatial constraint*. The cardio-thoracic ratio is driven by the cardiac silhouette, which lies largely outside the pure lung mask that gates editing. A system whose background lock is working should

therefore leave CTR nearly unchanged — which is what the small absolute magnitudes (0.007 and 0.025 against group means near 0.43) show. The SCP deltas are smaller still, around 10^{-4} on a metric with a standard deviation near 10^{-3}; their statistical significance reflects the paired design and $n \approx 200$ rather than any clinically meaningful shift.

The honest summary is that the clinical proxy metrics are not currently discriminative enough to support conclusions about clinical realism. Section 2 explains why the SCP proxy in particular should be treated as a weak instrument.

3 Qualitative Counterfactuals

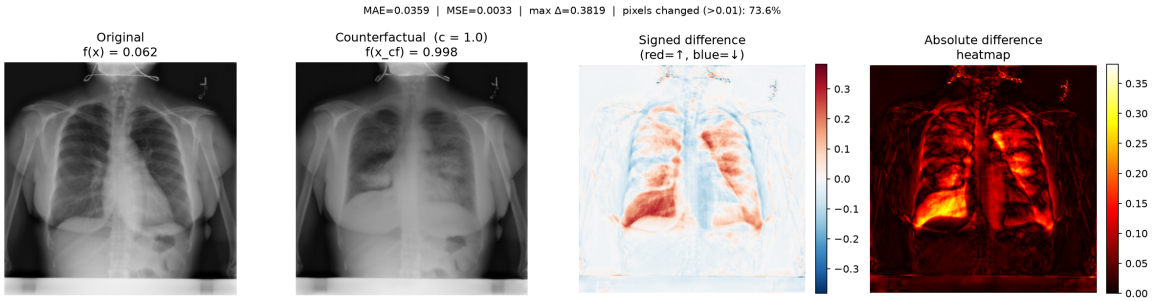


Figure 4.1: Normal → pneumonia. Left to right: original, counterfactual targeting $c = 1$, signed pixel difference, and absolute-difference heatmap, produced by `src/diff_viz.py`. The classifier score crosses the decision boundary by a wide margin.

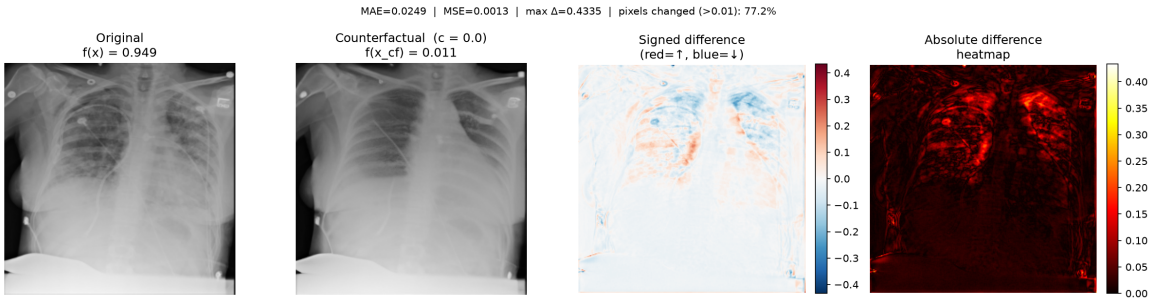


Figure 4.2: Pneumonia → normal. Same four-panel layout. Change is distributed over both lung fields, consistent with opacity removal, and at a lower overall magnitude than the n→p case — MAE 0.029 versus 0.042, as measured over 200 images per direction in `heldout_validation.json`.

Gross anatomical layout outside the lung field is visibly retained in both directions, which is the qualitative signature of the background lock holding as designed.

4 Training Diagnostics

The guidance diagnostics serialised alongside each policy checkpoint show that the two directions converged to visibly different control strategies. The n→p policy settles at a guidance bias of 1.298 — essentially the top of its $[0.70, 1.30]$ range — meaning it maximally favours classifier guidance over preservation, and suppresses preservation to a factor of 0.771. The p→n policy does the opposite, settling at 0.701, the bottom of the range, which *amplifies* preservation by $1.428\times$. Average reward differs accordingly: 21.41 for n→p against 14.87 for p→n.

Both policies pin their guidance bias to opposite ends of the available range, with standard deviations below 0.006. This near-deterministic saturation indicates the action bound, not the policy, is

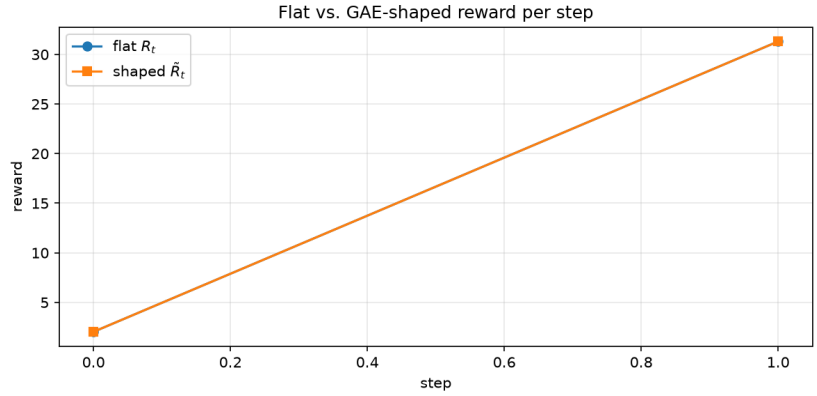


Figure 4.3: Flat per-step reward R_t against the GAE-shaped reward $\tilde{R}_t$ over a short post-training rollout (`factory/evaluate.py`, `plot_reward_trace`). The trace is short because this rollout reaches validity almost immediately, and the two curves are near-coincident — a direct consequence of PPO's per-update advantage normalisation absorbing a uniform shaping term. This is a training-health diagnostic, not a convergence curve.

the binding constraint on this dimension — a concrete signal that the range itself should be widened in future work.

III The Held-Out Validation Result

The central experimental finding of this internship is negative, and it concerns the interpretation of Table 4.1.

Counterfactual Validity there is computed with the same classifier that generated the guidance gradient. To test whether the edits carry disease signal rather than a perturbation tailored to that one model, job **17649721** re-scored 200 counterfactuals per direction with two TorchXRayVision DenseNet-121 classifiers that had never participated in guidance.

Table 4.3: Held-out validation (`checkpoints/heldout/heldout_validation.json`), $n = 200$ per direction, $\tau = 0.8$. Guidance classifier at generation time: `densenet121-res224-rsna` alone. Strict CV is the fraction of counterfactuals each validator independently judges valid.

Classifier	Role	CV n→p	CV p→n	Agreement
`-rsna`	guidance (circular)	0.995	0.885	—
`-chex`	held out	0.080	0.000	0.707
`-nih`	held out	0.000	0.000	0.697

The gap is stark. Counterfactuals that the guidance classifier accepts 99.5% of the time are accepted by an independent classifier at most 8% of the time, and in three of the four held-out cells, never. Transfer ratios — the fraction of the guidance model's score shift that the validators reproduce — are 0.205 and 0.590 for n→p and only 0.132 and 0.143 for p→n.

The agreement control is what makes this conclusive. On the *original*, unedited images the validators agree with the guidance classifier around 70% of the time (0.697 and 0.707), confirming they share a working notion of the task. They are not failing to recognise pneumonia in general; they are failing to recognise it in these specific generated images.

The conclusion is direct: **the p→n direction in particular was not removing disease, it was**

fooling $f(\cdot)$. The edits were largely adversarial perturbations against the guidance classifier rather than semantic changes to depicted pathology.

The edit-concentration figures recorded in the same artefact corroborate this. Concentration — the share of total absolute change carried by the top 5% of pixels — was 0.282 (n→p) and 0.297 (p→n) against a uniform baseline of 0.05, with 66% and 62% of all pixels changed and MAE of 0.042 and 0.029. A focal consolidation should concentrate well above 0.5. These edits have a concentrated core but carry most of their mass in a diffuse, low-amplitude tail spread over the majority of the image — the visual signature of an adversarial perturbation rather than a lesion.

This result motivated the three countermeasures described in Chapter 3: ensemble guidance, the Grad-CAM action-space constraint, and the annealed sparsity reward.

1 Status of the Countermeasures

The countermeasures are **implemented and enabled** in `config.py`, and a training run under the new configuration was in progress at the close of the internship: the most recent log (job **17653310**) shows the ensemble classifier loading correctly with both members, the Grad-CAM mask active, and the sparsity ramp resolved over updates $[20, 60)$ of 100.

A re-run of the held-out validation against the new configuration was not completed within the internship period. Whether ensemble guidance, spatial constraint and sparsity pressure actually close the transfer gap in Table 4.3 is therefore an open question, not a demonstrated result. It is the first item of future work in Section V.

The one measurement available from the new configuration is a caution rather than a result: the pipeline log notes that Grad-CAM is degenerate for one of the two ensemble members, so the constraint mask is effectively being computed from a single member. The ensemble-aware CAM path handles this by averaging over the remaining members instead of failing, but it means the spatial constraint is currently weaker than the design intends.

IV Strengths and Limitations

1 Strengths

- **A genuine spatial guarantee.** The background lock is a structural constraint rather than a penalty term. Content outside the lung field cannot be repainted regardless of the guidance gradient's behaviour, and the small CTR deltas in Table 4.2 are consistent with it holding.
- **Parameter-efficient adaptation.** Domain adaptation of an 866M-parameter generator was achieved by training 0.78% of its parameters, which is what made the project feasible within a single-GPU budget.
- **Sampling control without weight modification.** The PPO agent never touches the diffusion weights, keeping the action space interpretable at 9 bounded dimensions and the learned policy reusable across arbitrary query images.
- **Falsifiable evaluation.** The project's most valuable output is a control that exposed its own primary metric as circular. The held-out validation apparatus, its agreement control, and the disjointness rule between guidance and validator sets are reusable well beyond this system.

2 Limitations

- **Transfer to independent classifiers is unresolved.** This is the principal limitation. As of the end of the internship the counterfactuals demonstrably did not transfer (Section III), and the countermeasures are implemented but unvalidated (Section 1).
- **FOP is not computable.** The reference metric requires an unavailable detector, and the IP substitute measures a related but not identical property.
- **SCP is a weak proxy.** It is derived from a COCO-pretrained YOLOv8 feature norm rather than a dedicated costophrenic-recess detector. Its near-zero deltas in Table 4.2 should not be read as clinical measurements.
- **FID estimates are imprecise and high.** $N \approx 200$ against a 1024-dimensional feature space gives wide error bars, and the absolute values indicate a real realism gap.
- **Compute-constrained training scale.** The episode and epoch budgets are documented in `config.py` as conservative placeholders sized for the available walltime, not values established by ablation.
- **Two independently trained policies.** A single target-conditioned policy would let the harder direction share representation capacity with the easier one, but requires an environment refactor not undertaken here.
- **No baseline comparison was performed.** No GAN-based counterfactual baseline was implemented or evaluated within this internship, so the system's performance is not positioned against prior work. This is stated explicitly because it is a real gap in the evaluation, not an oversight in reporting.
- **Single dataset, single pathology.** All results concern binary pneumonia classification on RSNA. Nothing here establishes generalisation to other pathologies, other corpora, or other acquisition protocols.

V Perspectives and Future Work

- **Complete the validation loop (highest priority).** Re-run `heldout_validation.py` against the ensemble-guided, Grad-CAM-constrained, sparsity-annealed configuration and compare transfer ratios directly with Table 4.3. Every other item is subordinate to this one: without it, the countermeasures remain untested hypotheses.
- **Run the ablation arms.** The `SMOKE_ARM` infrastructure (`baseline` / `mask` / `all`) is already in place to attribute any improvement to the individual mechanisms rather than to their combination.
- **Repair the degenerate ensemble CAM.** With Grad-CAM degenerate for one ensemble member, the spatial constraint is operating at half strength; diagnosing this would strengthen the mechanism most directly aimed at the adversarial failure mode.
- **Widen the guidance-bias action range.** Both policies saturate at opposite ends of $[0.70, 1.30]$ with near-zero variance, indicating the bound rather than the policy is binding.
- **Increase evaluation sample size.** The single most direct lever on FID precision.

- **Implement a baseline comparison.** A GAN-based counterfactual generator evaluated under an identical protocol would place these results in context.
- **Unify the two policies.** A single target-conditioned policy with randomised per-episode target would be architecturally cleaner and would let the harder n→p direction borrow capacity from the easier one.
- **Broaden scope.** Extending beyond binary pneumonia to multi-pathology counterfactuals, and to a second corpus, would test whether any of this generalises.

VI Conclusion

This chapter has reported the system's measured performance and subjected it to an independent control that substantially qualifies it. High counterfactual validity against the guidance classifier proved not to survive contact with classifiers that never guided it. The countermeasures this motivated are built and running, but their effectiveness is not yet established. The following general conclusion reflects on what the internship produced and what it taught.

General Conclusion

This research internship at the Maersk Mc-Kinney Moller Institute produced a complete, reproducible pipeline for generating visual counterfactual explanations on chest radiographs: RSNA data ingestion, a Stable Diffusion 1.5 generator adapted to the radiograph domain through LoRA at 0.78% of its parameters, a gradient-guided DDIM sampler with a hard segmentation-gated background lock, two directional PPO policies controlling that sampler step by step, and an evaluation suite following the protocol of Singla *et al.* — all executable as batch jobs on an offline HPC cluster.

Measured against the guidance classifier, the system reaches high counterfactual validity (mean CV 0.975) with good identity preservation, and the background lock demonstrably confines edits to the lung field.

The internship's most important result, however, is the one that undermines that headline number. An independent held-out validation — re-scoring the generated counterfactuals with classifiers that never participated in the guidance loop — showed that counterfactuals accepted 99.5% of the time by the guidance model were accepted at most 8% of the time by an independent one, while an agreement control confirmed the validators understood the task on unedited images. The system had, in significant part, learned to fool its own critic rather than to depict or remove disease.

That finding reframed the second half of the internship. Ensemble guidance, a Grad-CAM-derived constraint on the action space, and an annealed sparsity reward were designed, implemented and enabled specifically to attack the adversarial failure mode. Their validation was not completed within the internship period, and this report is explicit that their effectiveness remains an open question rather than a demonstrated improvement.

Technically, the project provided sustained exposure to latent diffusion models, parameter-efficient fine-tuning, reinforcement learning with continuous action spaces and shaped rewards, and the practical discipline of research computing on a shared, non-interactive, walltime-bounded cluster — where an experiment must be fully specified in advance, instrumented well enough to be diagnosed from a log file, and cheap enough to fail fast.

The methodological lesson runs deeper than any of those techniques. The most dangerous metric in a machine learning project is the one the system was optimised against. Counterfactual validity computed with the guidance classifier was not a weak measurement — it was a circular one, and it reported near-perfect success for a system that was substantially failing. Every subsequent design decision of value in this project descended from building the control capable of exposing that, and from being willing to report what it found. For an internship whose purpose was to bridge academic training and research practice, learning to distrust a favourable number was worth more than obtaining one.

Glossary

Adversarial perturbation A small, often visually diffuse change to an image that flips a classifier's prediction without altering the semantic content a human would recognise. Distinguishing genuine counterfactuals from adversarial perturbations is the central validation problem of this project.

Background lock A hard spatial constraint, adapted from RePaint and Blended Latent Diffusion, in which the latent outside a permitted edit mask is overwritten at every denoising step with the original image's forward-noised latent, guaranteeing that region cannot be repainted.

Counterfactual explanation For a query image x classified by f, an image x_c that f assigns to a different target class while remaining realistic and preserving the identity of x. The difference between x and x_c constitutes the explanation.

Counterfactual Validity (CV) The fraction of generated counterfactuals for which the classifier's score both moves by more than a margin τ and crosses the decision boundary in the intended direction.

CTR (Cardio-Thoracic Ratio) The ratio of the heart's horizontal span to the thorax's, computed here from segmentation masks. A ratio above 0.5 is the standard clinical cardiomegaly threshold.

DDIM Denoising Diffusion Implicit Model: a deterministic, non-Markovian sampling procedure for diffusion models that permits partial forward-then-reverse trajectories, and therefore image editing rather than only synthesis from noise.

FID (Fréchet Inception Distance) A distributional realism measure comparing the means and covariances of deep features extracted from real and generated image sets. Lower is better. Computed here in TorchXRayVision DenseNet feature space rather than ImageNet Inception space.

FOP (Foreign-Object Preservation) A metric measuring whether medical hardware — tubes, leads, pacemakers — survives counterfactual editing. Requires a dedicated device detector, and is not computable offline in this project.

GAE (Generalized Advantage Estimation) A variance-reduction technique for policy-gradient methods that interpolates between one-step and Monte-Carlo advantage estimates via a parameter λ.

Grad-CAM Gradient-weighted Class Activation Mapping: a technique producing class-discriminative localisation maps by weighting convolutional feature maps with the gradients flowing into them. Used here both as a diagnostic and as a spatial constraint.

Guidance The injection of a classifier's gradient into a diffusion model's sampling trajectory, steering generation toward a target class without retraining the generator.

Held-out validation Re-scoring generated counterfactuals with classifiers that never participated in the guidance loop, to distinguish genuine semantic change from adversarial perturbation against the guidance model.

Identity Preservation (IP) A model-free substitute for FOP used here, combining MS-SSIM (global anatomical similarity) with the correlation of high-pass-filtered images (survival of thin, high-contrast structures).

Latent diffusion A diffusion model operating in the compressed latent space of a pretrained autoencoder rather than in pixel space, reducing computational cost by orders of magnitude.

LoRA (Low-Rank Adaptation) A parameter-efficient fine-tuning technique that freezes pretrained weights and trains small injected low-rank matrix pairs, drastically reducing the memory and compute required to adapt a large model.

MS-SSIM Multi-Scale Structural Similarity Index Measure: an image similarity metric evaluating luminance, contrast and structure across multiple spatial scales.

PPO (Proximal Policy Optimization) A policy-gradient reinforcement learning algorithm that constrains each update with a clipped surrogate objective, trading some theoretical rigour for stability and sample efficiency.

Reward shaping The addition of auxiliary terms to a reinforcement learning reward to guide learning toward desired behaviour. A recurring source of bugs in this project, since the terms' relative magnitudes — not their presence — determine what the agent actually optimises.

SCP (Costophrenic-recess score) A proxy for pleural-effusion-related blunting of the costophrenic angle, approximated here from an object detector's feature norm in a lower-lobe crop.

SLURM Simple Linux Utility for Resource Management: the batch scheduler allocating compute nodes on the HPC cluster used for all experiments.

Transfer ratio The fraction of the guidance classifier's score shift that an independent held-out classifier reproduces on the same counterfactual. Low values indicate an adversarial rather than semantic edit.

VAE (Variational Autoencoder) The encoder-decoder pair mapping images to and from the compressed latent space in which the diffusion model operates. Frozen throughout this project.

Bibliography

[1] S. Singla, M. Eslami, B. Pollack, S. Wallace, and K. Batmanghelich. *Explaining the Black-box Smoothly — A Counterfactual Approach.* Medical Image Analysis, vol. 84, 2022.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. *Proximal Policy Optimization Algorithms.* arXiv:1707.06347, 2017.

[3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. *High-Dimensional Continuous Control Using Generalized Advantage Estimation.* ICLR, 2016.

[4] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. *High-Resolution Image Synthesis with Latent Diffusion Models.* CVPR, 2022.

[5] J. Song, C. Meng, and S. Ermon. *Denoising Diffusion Implicit Models.* ICLR, 2021.

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR, 2022.

[7] A. Lugmayr, M. Danelljan, A. Romero, F. Yu, R. Timofte, and L. Van Gool. *RePaint: Inpainting using Denoising Diffusion Probabilistic Models.* CVPR, 2022.

[8] O. Avrahami, D. Lischinski, and O. Fried. *Blended Diffusion for Text-driven Editing of Natural Images.* CVPR, 2022.

[9] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.* ICCV, 2017.

[10] J. P. Cohen, J. D. Viviano, P. Bertin, P. Morrison, P. Torabian, M. Guarrera, M. P. Lungren, A. Chaudhari, R. Brooks, M. Hashir, and H. Bertrand. *TorchXRayVision: A library of chest X-ray datasets and models.* MIDL, 2022.

[11] Radiological Society of North America. *RSNA Pneumonia Detection Challenge.* Kaggle, 2018. https://www.kaggle.com/c/rsna-pneumonia-detection-challenge

[12] Z. Wang, E. P. Simoncelli, and A. C. Bovik. *Multiscale Structural Similarity for Image Quality Assessment.* Asilomar Conference on Signals, Systems and Computers, 2003.

[13] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium.* NeurIPS, 2017.

[14] G. Jocher, A. Chaurasia, and J. Qiu. *Ultralytics YOLOv8.* 2023. https://github.com/ultralytics/ultralytics

[15] A. B. Yoo, M. A. Jette, and M. Grondona. *SLURM: Simple Linux Utility for Resource Management.* Job Scheduling Strategies for Parallel Processing, 2003.